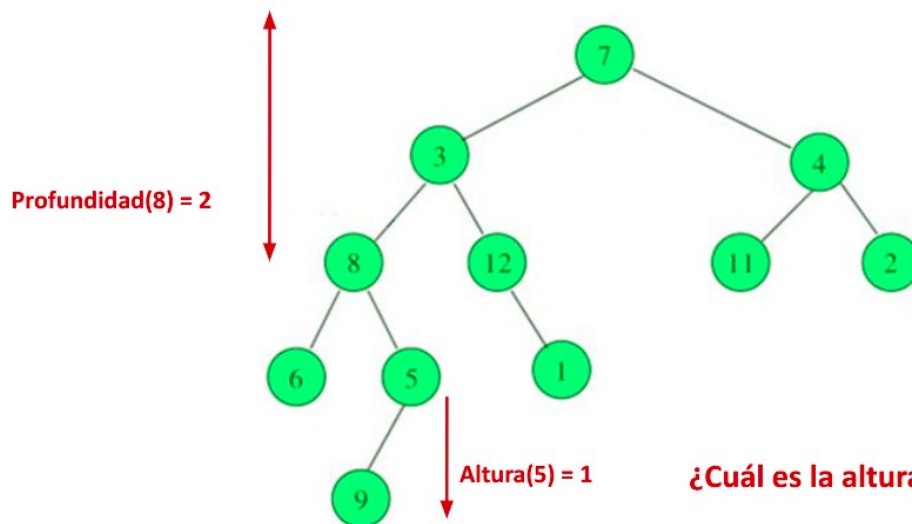




¿Cuál es la altura del nodo 8?

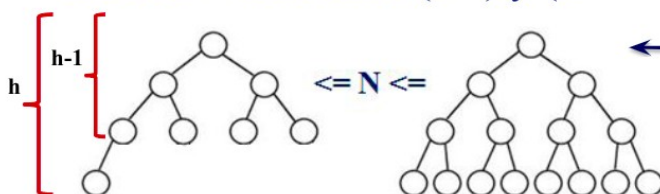
¿Cuál es la profundidad del nodo 12?

¿Cuáles son los ancestros del nodo 11?

- **Árbol binario completo:** Dado un árbol binario T de altura h , diremos que T es completo si es lleno de altura $h-1$ y el nivel h se completa de izquierda a derecha.

- **Cantidad de nodos en un árbol binario completo:**

Sea T un árbol binario completo de altura h , la cantidad de nodos N varía entre (2^h) y $(2^{h+1} - 1)$



- Si el árbol es lleno
 $N = (2^{h+1} - 1)$

- Si no, el árbol es lleno en la altura $h-1$ y tiene por lo menos un nodo en el nivel h :
 $N = (2^{h-1+1} - 1) + 1 = (2^h - 1 + 1)$

1) Construcción de un árbol de expresión a partir de una expresión postfija

Algoritmo:

tomo un carácter de la expresión

mientras (existe carácter) hacer

*si es un **operando** -> creo un nodo y lo apilo.*

*si es un **operador** (lo tomo como la raíz de los dos últimos nodos creados)*

*-> - **creo** un nodo R,*

*- **desapilo** y lo agrego como hijo derecho de R*

*- **desapilo** y lo agrego como hijo izquierdo de R*

*- **apilo** R.*

tomo otro carácter

fin

2) Construcción de un árbol de expresión a partir de una expresión prefija

Algoritmo:

ArbolExpresión (A: ArbolBin, exp: string)

si exp nulo -> nada.

si es un operador -> - creo un nodo raíz R

- ArbolExpresión (subArblzq de R, exp

(sin 1° carácter))

- ArbolExpresión (subArbDer de R, exp

(sin 1° carácter))

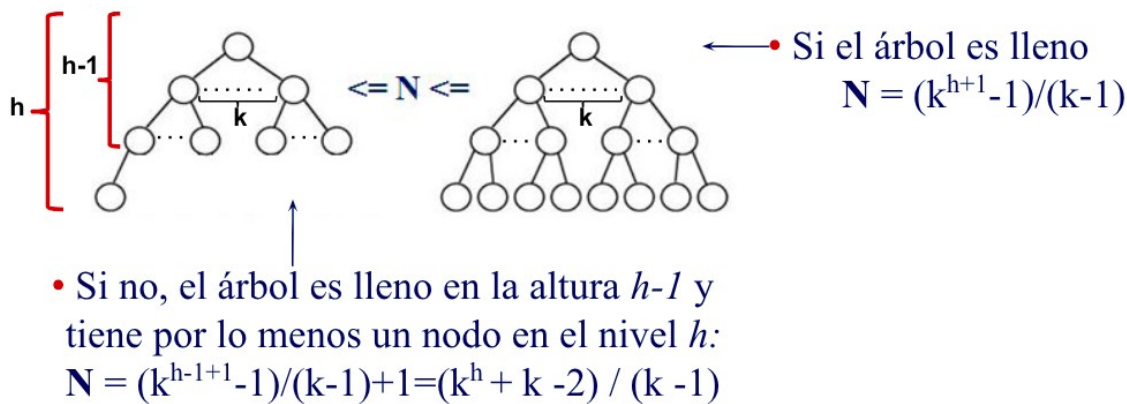
si es un operando -> creo un nodo (hoja)

- **Árbol completo:** Dado un árbol T de grado k y altura h , diremos que T es *completo* si es lleno de altura $h-1$ y el nivel h se completa de izquierda a derecha.
- **Cantidad de nodos en un árbol lleno:**
Sea T un árbol lleno de grado k y altura h , la cantidad de nodos N es $(k^{h+1} - 1) / (k-1)$ ya que:

Descripción y terminología

- **Cantidad de nodos en un árbol completo:**

Sea T un árbol completo de grado k y altura h , la cantidad de nodos N varía entre $(k^h + k - 2) / (k-1)$ y $(k^{h+1} - 1) / (k-1)$ ya que ...



- ✓ Lista ordenada
 - Insert tiene $O(N)$ operaciones
 - DeleteMin tiene $O(1)$ operaciones
- ✓ Lista no ordenada
 - Insert tiene $O(1)$ operaciones
 - DeleteMin tiene $O(N)$ operaciones
- ✓ Árbol Binario de Búsqueda
 - Insert y DeleteMin tienen en promedio $O(\log N)$ operaciones

Heap Binaria

- Es una implementación de colas de prioridad que no usa punteros y permite implementar ambas operaciones con $O(\log N)$ operaciones en el peor caso
- Cumple con dos propiedades:
 - ✓ Propiedad estructural
 - ✓ Propiedad de orden

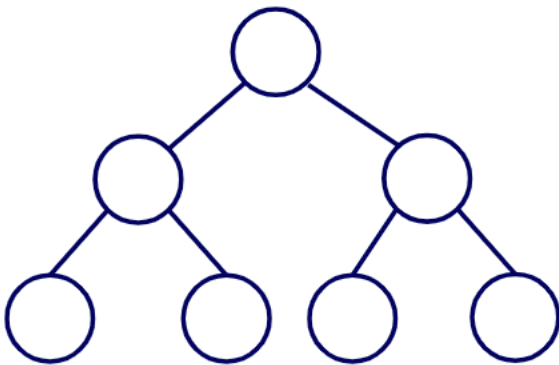
Propiedad estructural

Una heap es un árbol binario completo

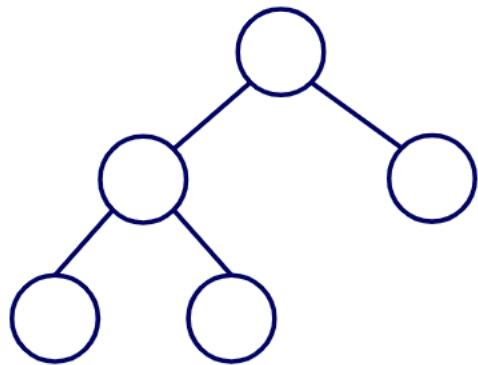
- ✓ En un árbol binario lleno de altura h , los nodos internos tienen exactamente 2 hijos y las hojas tienen la misma profundidad
- ✓ Un árbol binario completo de altura h es un árbol binario lleno de altura $h-1$ y en el nivel h , los nodos se completan de izquierda a derecha

Propiedad estructural (cont.)

Árbol binario lleno



Árbol binario completo



Propiedad estructural (cont.)

- ✓ El número de nodos n de un árbol binario completo de altura h , satisface:

$$2^h \leq n \leq (2^{h+1}-1)$$

Demostración:

- Si el árbol es lleno, $n = 2^{h+1}-1$
- Si no, el árbol es lleno en la altura $h-1$ y tiene por lo menos un nodo en el nivel h :

$$n = 2^{h-1+1}-1+1 = 2^h$$

La altura h del árbol es de **$O(\log n)$**

Propiedad estructural (cont.)

- Dado que un árbol binario completo es una estructura de datos regular, puede almacenarse en un arreglo, tal que:

- ✓ La raíz está almacenada en la posición 1
- ✓ Para un elemento que está en la posición i :
 - El hijo izquierdo está en la posición $2*i$
 - El hijo derecho está en la posición $2*i + 1$
 - El padre está en la posición $\lfloor i/2 \rfloor$

Propiedad de orden

□ MinHeap

- El elemento mínimo está almacenado en la raíz
- El dato almacenado en cada nodo es menor o igual al de sus hijos

□ MaxHeap

- Se usa la propiedad inversa

Implementación de Heap

Una heap H consta de:

- *Un arreglo que contiene los datos*
- *Un valor que me indica el número de elementos almacenados*

Ventaja:

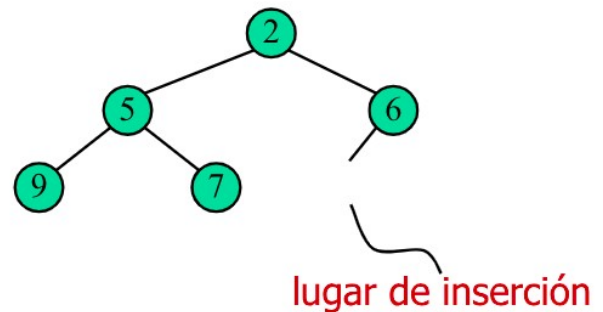
- ✓ No se necesita usar punteros
- ✓ Fácil implementación de las operaciones

Operación: Insert

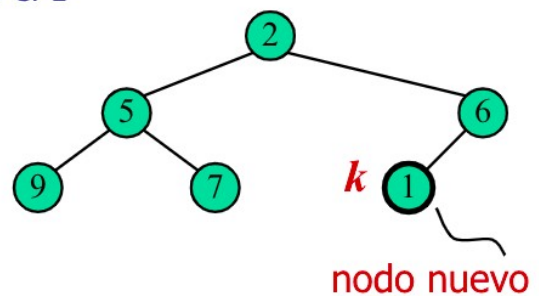
□ El dato se inserta como último ítem en la heap

□ La propiedad de la heap puede ser violada

□ Se debe hacer un filtrado hacia arriba para restaurar la propiedad de orden

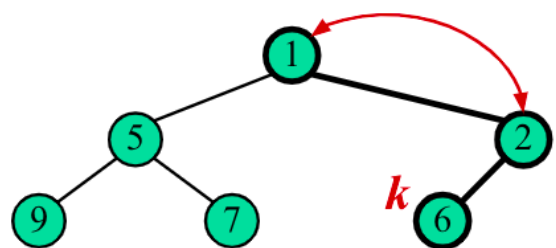
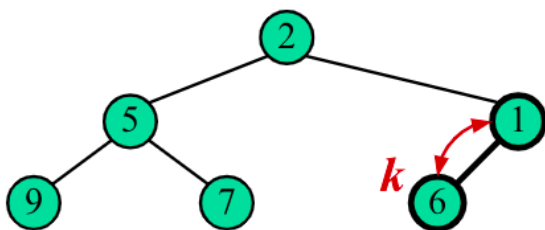


Inserto el 1



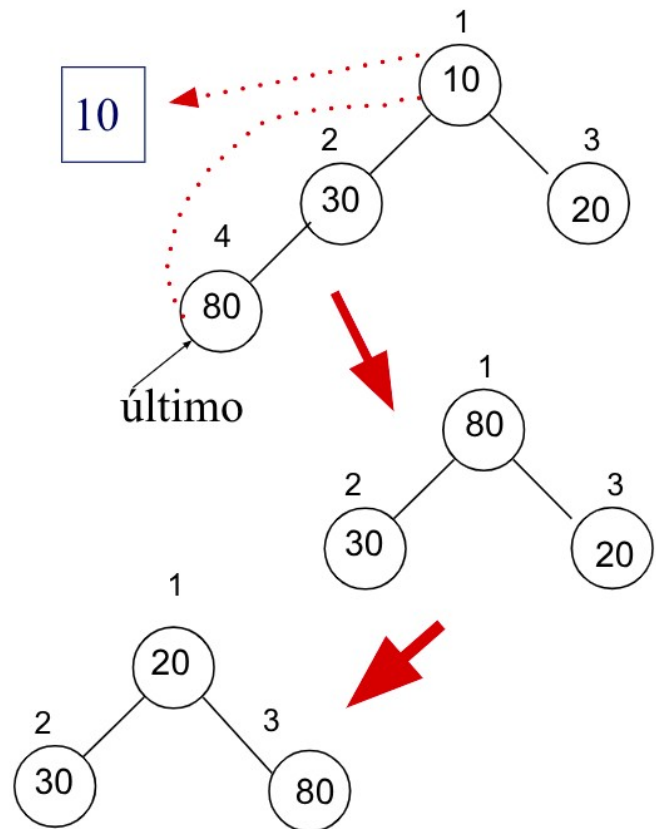
Insert: Filtrado hacia arriba (Percolate Up)

- El filtrado hacia arriba restaura la propiedad de orden intercambiando *k* a lo largo del camino hacia arriba desde el lugar de inserción
- El filtrado termina cuando la clave *k* alcanza la raíz o un nodo cuyo padre tiene una clave menor
- Ya que el algoritmo recorre la altura de la heap, tiene $O(\log n)$ intercambios



Operación: DeleteMin

- ❑ Guardo el dato de la raíz
- ❑ Elimino el último elemento y lo almaceno en la raíz
- ❑ Se debe hacer un filtrado hacia abajo para restaurar la propiedad de orden



DeleteMin: Filtrado hacia abajo (Percolate Down)

- ❑ Es similar al filtrado hacia arriba
- ❑ El filtrado hacia abajo restaura la propiedad de orden intercambiando el dato de la raíz hacia abajo a lo largo del camino que contiene los hijos mínimos
- ❑ El filtrado termina cuando se encuentra el lugar correcto dónde insertarlo
- ❑ Ya que el algoritmo recorre la altura de la heap, tiene $O(\log n)$ operaciones de intercambio.

Otras operaciones

□ DecreaseKey(x , Δ , H)

- Decrementa la clave que está en la posición x de la heap H , en una cantidad Δ

□ IncreaseKey(x , Δ , H)

- Incrementa la clave que está en la posición x de la heap H , en una cantidad Δ

□ DeleteKey(x)

- Elimina la clave que está en la posición x

- Puede realizarse:

